Rapport d'atelier MySQL

Conception et exploitation de la base de données
atelier_commerce

Travail à rendre : rapport contenant les captures d'écran prouvant les exécutions

Novembre 2026

Introduction

Ce rapport décrit, étape par étape, la réalisation de la base de données **atelier_commerce** sur un serveur MySQL, avec MySQL Workbench comme client. Chaque étape est illustrée par une capture d'écran de la requête exécutée et de son résultat, conformément à la consigne de l'atelier.

La base modélise un petit système de vente : des **clients** passent des **commandes**, chaque commande contient des lignes de **détails** portant sur des **produits**, et les commandes sont règlées par des **paiements**.

Plan du travail

#	Étape	Notion MySQL
1	Création de la base et des tables	CREATE DATABASE, CREATE TABLE, clés étrangères
2	Insertion des données	INSERT INTO
3	Jointures	INNER, LEFT, RIGHT JOIN, UNION
4	Agrégations	GROUP BY, SUM, AVG, HAVING
5	Fonctions IF(), DATEDIFF() et LIMIT	IF, DATEDIFF, ORDER BY, LIMIT
6	Sous-requêtes	IN, NOT IN, EXISTS
7	Index et EXPLAIN	CREATE INDEX, EXPLAIN
8	Vue	CREATE VIEW, SELECT sur la vue
9	Droits utilisateurs	CREATE USER, GRANT, SHOW GRANTS, REVOKE

Étape 1 - Création de la base et des tables

1.1 Création de la base de données

J'ai d'abord créé la base **atelier_commerce**, puis je l'ai sélectionnée avec **USE** afin que toutes les commandes suivantes s'appliquent à cette base.

```
CREATE DATABASE atelier_commerce;
USE atelier_commerce;
```



Figure 1 - Création et sélection de la base atelier_commerce

1.2 Création des tables

J'ai créé cinq tables. Les tables **clients** et **produits** sont indépendantes. Les trois autres sont reliées par des **clés étrangères** qui garantissent l'intégrité des données :

- **clients** (id, nom, ville) - clé primaire auto-incrémentée ;
- **produits** (id, nom_produit, prix_unitaire DECIMAL(10,2)) ;
- **commandes** (id, client_id, date_commande) - client_id référence clients(id) ;
- **details_commandes** (id, commande_id, produit_id, quantite) - référence commandes(id) et produits(id) ;

- **paiements** (id, commande_id, montant, mode_paiement, date_paiement) - référence commandes(id).

```sql
CREATE TABLE clients(
    id INT auto_increment PRIMARY KEY,
    nom varchar(50),
    ville varchar(50)
);

CREATE TABLE produits(
    id INT auto_increment PRIMARY KEY,
    nom_produit varchar(50),
    prix_unitaire decimal(10,2)
);

CREATE TABLE commandes(
    id INT auto_increment PRIMARY KEY,
    client_id INT,
    date_commande DATE,
    FOREIGN KEY (client_id) REFERENCES clients(id)
);
```

Figure 2 - Création des tables clients, produits et commandes

```sql
CREATE TABLE details_commandes(
    id INT auto_increment PRIMARY KEY,
    commande_id INT,
    produit_id INT,
    quantite INT,
    FOREIGN KEY (commande_id) REFERENCES commandes(id),
    FOREIGN KEY (produit_id) REFERENCES produits(id)
);

CREATE TABLE paiements(
    id INT AUTO_INCREMENT PRIMARY KEY,
    commande_id INT,
    montant DECIMAL(10,2),
    mode_paiement VARCHAR(30),
    date_paiement DATE,
    FOREIGN KEY (commande_id) REFERENCES commandes(id)
);
```

Figure 3 - Création des tables details_commandes et paiements

L'explorateur d'objets de MySQL Workbench confirme que les cinq tables existent bien dans la base.

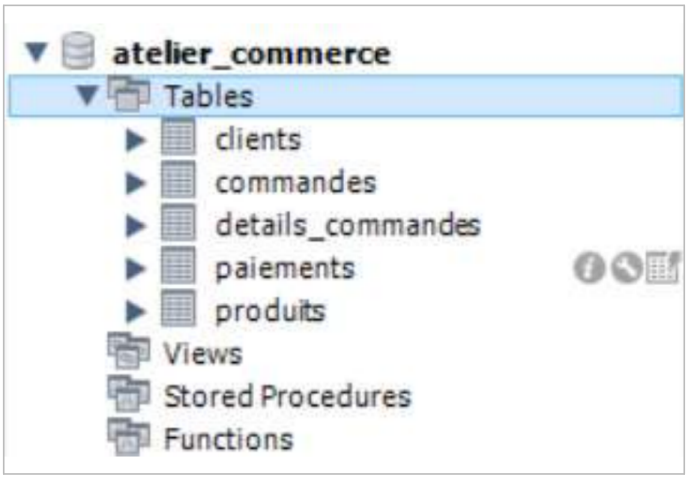


Figure 4 - Les cinq tables visibles dans l'explorateur de la base

Étape 2 - Insertion des données

J'ai ensuite alimenté chaque table avec un jeu de données de test : 4 clients (Alice, Bob, Charlie, Dina), 4 produits (Ordinateur, Souris, Clavier, Écran), 3 commandes, leurs lignes de détail et 3 paiements. Les tables parentes sont remplies avant les tables enfants pour respecter les clés étrangères. Dina n'a volontairement aucune commande : cela servira à illustrer les jointures externes.

```
INSERT INTO clients(nom,ville) VALUES ('Alice','Paris'),('Bob','Lyon'),('Charlie','Toulouse'),('Dina','Nice');

INSERT INTO produits(nom_produit,prix_unitaire) VALUES ('Ordinateur',950), ('Souris',20), ('Clavier',45), ('Ecran',250);

INSERT INTO commandes(client_id,date_commande) VALUES (1,'2025-01-20'), (2,'2025-02-01'), (3,'2025-02-10');

INSERT INTO details_commandes(commande_id, produit_id, quantite) VALUES (1,1,1),(1,2,2),(2,3,1),(3,4,2),(4,2,3);

INSERT INTO paiements(commande_id,montant,mode_paiement,date_paiement) VALUES (1,990,'Carte','2025-01-12'),
        (2,45,'Especes','2025-01-21'),(3,500,'Virement','2025-02-05');
```

Figure 5 - Instructions INSERT INTO pour les cinq tables

Étape 3 - Jointures

3.1 INNER JOIN

Cette requête relie quatre tables (clients, commandes, details_commandes, produits) pour afficher, pour chaque ligne de commande, le client, le numéro de commande, le produit et la quantité. **INNER JOIN** ne conserve que les lignes qui ont une correspondance dans toutes les tables : Dina, sans commande, n'apparaît donc pas.

```
1
2 •  SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite FROM clients c
3                          INNER JOIN commandes cmd ON c.id=cmd.client_id
4                          INNER JOIN details_commandes dc ON cmd.id=dc.commande_id
5                          INNER JOIN produits p ON dc.produit_id=p.id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom	commande	nom_produit	quantite
Alice	1	Ordinateur	1
Alice	1	Souris	2
Bob	2	Clavier	1
Charlie	3	Ecran	2

Figure 6 - INNER JOIN sur quatre tables et son résultat (4 lignes)

3.2 LEFT JOIN

Avec **LEFT JOIN**, tous les clients sont conservés, même sans commande. Dina apparaît avec la valeur **NULL** dans la colonne num_commande, ce qui permet d'identifier les clients qui n'ont encore rien commandé.

```
2 •  SELECT c.nom, cmd.id AS num_commande FROM clients c
3        LEFT JOIN commandes cmd ON c.id=cmd.client_id;
4
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom	num_commande
Alice	1
Bob	2
Charlie	3
Dina	NULL

Figure 7 - LEFT JOIN clients / commandes : Dina apparaît avec NULL

3.3 RIGHT JOIN

RIGHT JOIN conserve toutes les commandes. Comme chaque commande a un client, le résultat contient 3 lignes et Dina est absente.

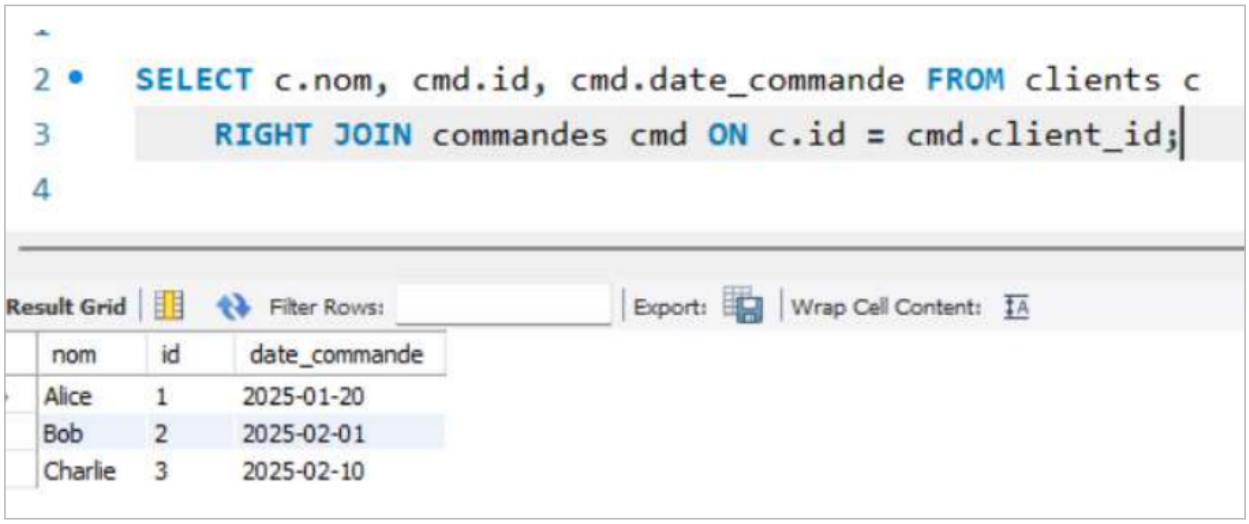


Figure 8 - RIGHT JOIN clients / commandes

3.4 FULL JOIN simulé avec UNION

MySQL ne dispose pas de FULL OUTER JOIN. Je l'ai simulé en combinant un LEFT JOIN et un RIGHT JOIN avec **UNION**, qui supprime aussi les doublons. Le résultat contient à la fois les clients sans commande et toutes les commandes.

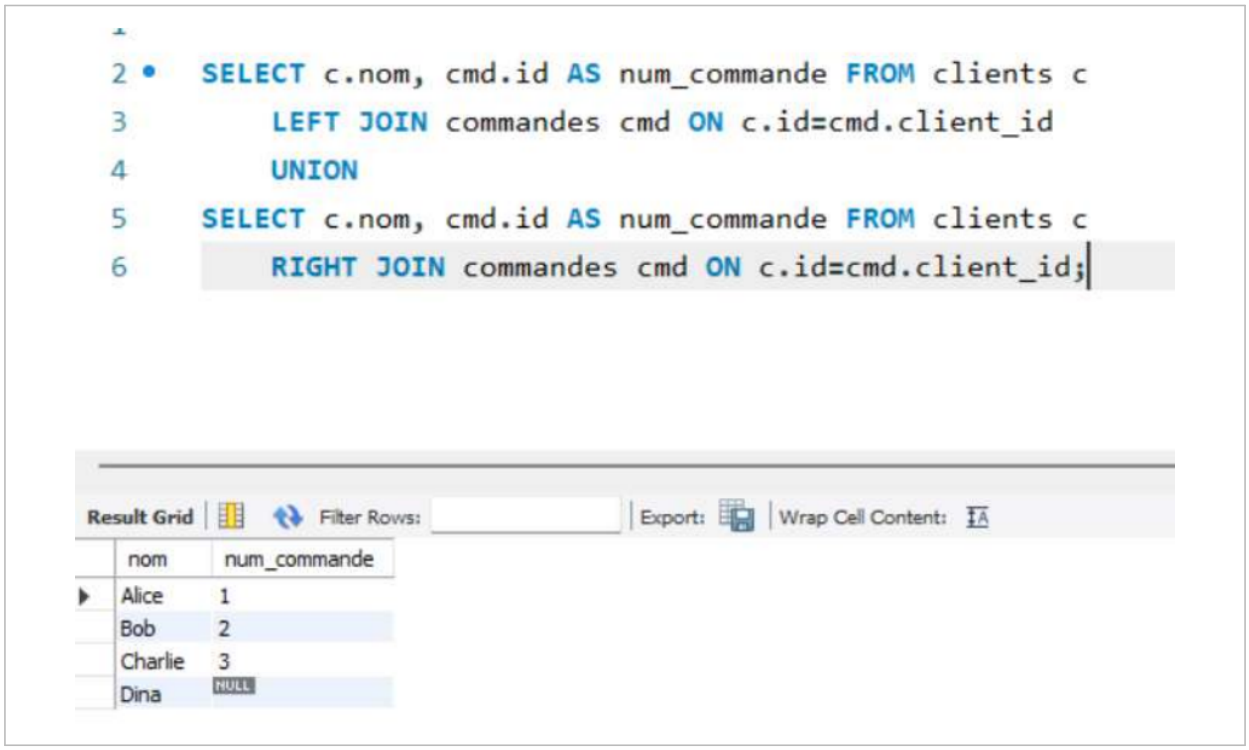


Figure 9 - Simulation d'un FULL JOIN par UNION

Étape 4 - Requêtes d'agrégation

4.1 GROUP BY avec SUM

Pour obtenir le total de chaque commande, je multiplie le prix unitaire par la quantité sur chaque ligne, puis je les additionne avec **SUM** en regroupant par commande avec **GROUP BY**. Résultat : commande 1 = 990, commande 2 = 45, commande 3 = 500.

```
2 •  SELECT cmd.id AS commande, sum(p.prix_unitaire*dc.quantite) AS total_commande FROM commandes cmd
3        JOIN details_commandes dc ON cmd.id=dc.commande_id
4        JOIN produits p ON dc.produit_id=p.id
5        GROUP BY cmd.id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

commande	total_commande
1	990.00
2	45.00
3	500.00

Figure 10 - Total par commande avec SUM et GROUP BY

4.2 HAVING

HAVING filtre les groupes après l'agrégation (contrairement à WHERE qui filtre avant). Ici, seuls les clients dont le total dépasse 500 sont retenus : seule Alice (990) satisfait la condition, Charlie étant à 500 exactement.

```
1
2 •  SELECT c.nom, sum(p.prix_unitaire*dc.quantite) AS total_client FROM clients c
3        JOIN commandes cmd ON c.id=cmd.client_id
4        JOIN details_commandes dc ON cmd.id=dc.commande_id
5        JOIN produits p ON dc.produit_id = p.id
6        GROUP BY c.nom HAVING total_client>500;
7
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom	total_client
Alice	990.00

Figure 11 - Clients dont le total dépasse 500 (GROUP BY + HAVING)

4.3 AVG

La fonction **AVG** calcule le prix moyen par produit. Chaque produit n'apparaissant qu'une fois dans la table, la moyenne est égale au prix unitaire (Ordinateur 950, Souris 20, Clavier 45, Écran 250).

```
1
2 •  SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen FROM produits p
3            GROUP BY p.nom_produit;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom_produit	prix_moyen
Ordinateur	950.000000
Souris	20.000000
Clavier	45.000000
Ecran	250.000000

Figure 12 - Prix moyen par produit avec AVG

Étape 5 - IF(), DATEDIFF() et LIMIT

5.1 Fonction IF()

IF(condition, valeur_si_vrai, valeur_si_faux) permet de créer une catégorie de client : si le total dépasse 800, le client est classé « Premium », sinon « Standard ». Alice (990) est Premium ; Bob (45) et Charlie (500) sont Standard.

```
1
2 •  SELECT c.nom, SUM(p.prix_unitaire*dc.quantite) AS total,
3        IF (SUM(p.prix_unitaire*dc.quantite)>800,'Premium','Standard') AS categorie
4        FROM clients c
5        JOIN commandes cmd ON c.id = cmd.client_id
6        JOIN details_commandes dc ON cmd.id=dc.commande_id
7        JOIN produits p ON dc.produit_id = p.id
8        GROUP BY c.nom;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom	total	categorie
Alice	990.00	Premium
Bob	45.00	Standard
Charlie	500.00	Standard

Figure 13 - Classification Premium / Standard avec IF()

5.2 Fonction DATEDIFF()

DATEDIFF(date1, date2) renvoie le nombre de jours entre deux dates. La requête calcule le délai entre la commande et son paiement en joignant paiements et commandes.

```
1
2 •  SELECT p.id AS paiement, cmd.id AS commande,
3        datediff(p.date_paiement, cmd.date_commande) AS delai_jours
4        FROM paiements p
5        JOIN commandes cmd ON p.commande_id = cmd.id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

paiement	commande	delai_jours

Figure 14 - Calcul du délai de paiement avec DATEDIFF()

5.3 ORDER BY et LIMIT

Pour obtenir le classement des meilleurs clients, je trie les totaux par ordre décroissant avec **ORDER BY ... DESC** et je ne garde que les deux premiers avec **LIMIT 2** : Alice (990) puis Charlie (500).

```sql
SELECT c.nom, SUM(p.prix_unitaire*dc.quantite) AS total FROM clients c
    JOIN commandes cmd ON c.id=cmd.client_id
    JOIN details_commandes dc ON cmd.id=dc.commande_id
    JOIN produits p ON dc.produit_id=p.id
    GROUP BY c.nom
    ORDER BY total DESC
    LIMIT 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

nom	total
Alice	990.00
Charlie	500.00

Figure 15 - Les 2 meilleurs clients avec ORDER BY DESC et LIMIT 2

Étape 6 - Sous-requêtes

J'ai écrit la même question - « quels clients ont une commande payée ? » - sous trois formes pour comparer les approches.

6.1 IN

Sous-requêtes imbriquées : on cherche les clients dont l'id figure parmi les client_id des commandes présentes dans la table paiements.

```sql
SELECT nom FROM clients WHERE id IN(
    SELECT client_id FROM commandes WHERE id IN(
        SELECT commande_id FROM paiements));
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom

Figure 16 - Sous-requêtes imbriquées avec IN

6.2 NOT IN

L'inverse : les clients qui ne figurent pas dans ce résultat.

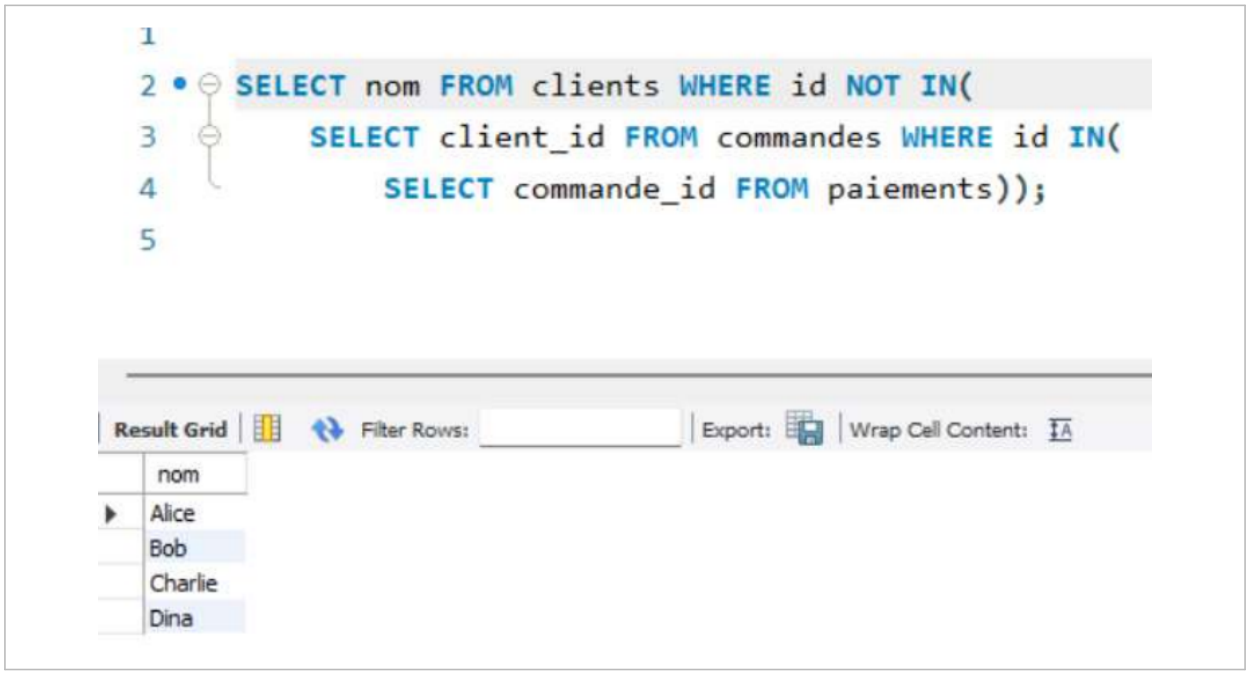


Figure 17 - Sous-requête avec NOT IN

6.3 EXISTS

EXISTS est une sous-requête corrélée : pour chaque client, MySQL vérifie s'il existe au moins une commande reliée à un paiement. Elle est souvent plus efficace que IN sur de gros volumes.

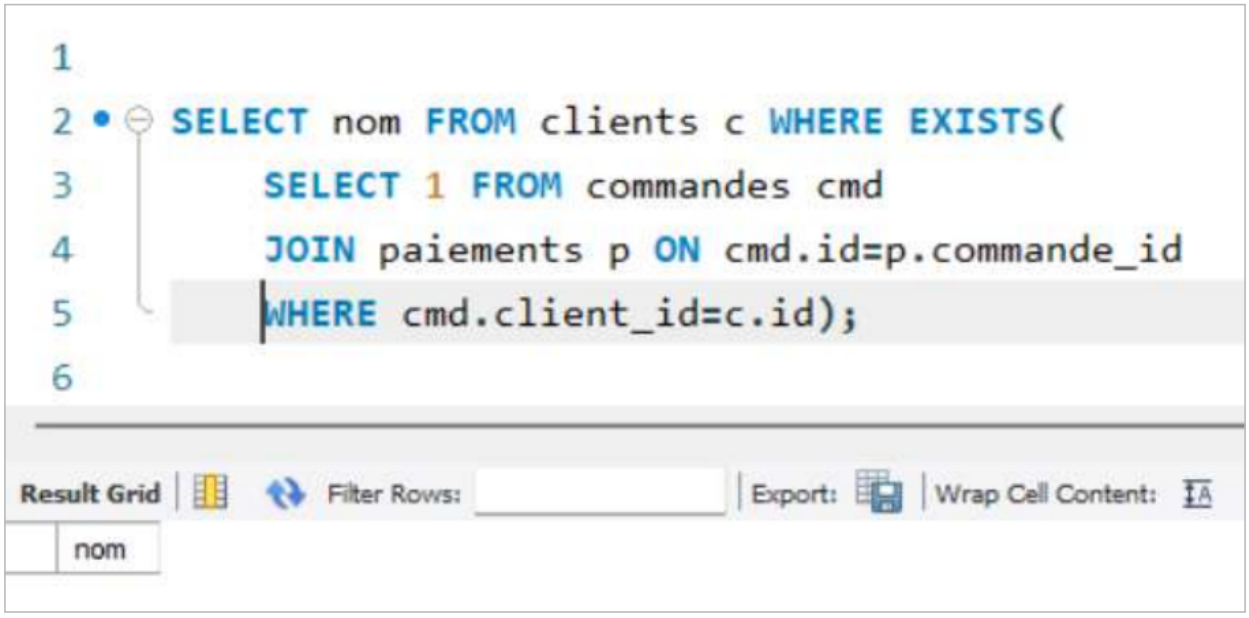


Figure 18 - Sous-requête corrélée avec EXISTS

Étape 7 - Index et vérification avec EXPLAIN

Un **index** accélère les recherches sur une colonne en évitant de parcourir toute la table. J'ai créé deux index : **idx_nom_produit** sur produits(nom_produit) et **idx_client_ville** sur clients(ville).

```sql
CREATE INDEX idx_nom_produit ON produits(nom_produit);
CREATE INDEX idx_client_ville ON clients(ville);
```

Figure 19 - Création des index idx_nom_produit et idx_client_ville

J'ai ensuite vérifié son effet avec **EXPLAIN**. Le plan d'exécution indique **type = ref**, **key = idx_nom_produit** et **rows = 1** : MySQL utilise bien l'index et n'examine qu'une seule ligne, au lieu d'un parcours complet de la table (type ALL).

```sql
EXPLAIN SELECT * FROM produits WHERE nom_produit='Souris';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	produits	NULL	ref	idx_nom_produit	idx_nom_produit	203	const	1	100.00	NULL

Figure 20 - EXPLAIN : l'index idx_nom_produit est utilisé

Étape 8 - Création et lecture d'une vue

Une **vue** est une requête enregistrée qui se consulte comme une table. J'ai créé **vue_ventes**, qui regroupe le client, le produit, la quantité, le montant calculé et le mode de paiement. Le **LEFT JOIN** sur paiements permet de garder les commandes même sans paiement associé.

```sql
CREATE VIEW vue_ventes AS
    SELECT c.nom, p.nom_produit, dc.quantite, (p.prix_unitaire*dc.quantite) AS montant, pa.mode_paiement
    FROM clients c
    JOIN commandes cmd ON c.id = cmd.client_id
    JOIN details_commandes dc ON cmd.id=dc.commande_id
    JOIN produits p ON dc.produit_id=p.id
    LEFT JOIN paiements pa ON cmd.id=pa.commande_id;
```

Figure 21 - Création de la vue vue_ventes

La vue est ensuite lue comme n'importe quelle table, ici avec un filtre sur les montants supérieurs à 200.

```
1
2 •  SELECT * FROM vue_ventes WHERE montant>200;
3
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

nom	nom_produit	quantite	montant	mode_paiement
Alice	Ordinateur	1	950.00	NULL
Charlie	Ecran	2	500.00	NULL

Figure 22 - SELECT sur la vue vue_ventes avec filtre

Étape 9 - Gestion des droits utilisateurs

Pour finir, j'ai appliqué le principe du moindre privilège en créant deux comptes aux droits différents :

- **vendeur** : droits SELECT, INSERT et UPDATE sur la base atelier_commerce ;
- **manager** : ALL PRIVILEGES sur la base atelier_commerce.

SHOW GRANTS permet de contrôler les droits du manager (USAGE global et ALL PRIVILEGES sur atelier_commerce). Enfin, **REVOKE** retire le droit UPDATE au vendeur, qui ne peut alors plus que consulter et ajouter des données.

```
1
2 •  CREATE USER 'vendeur'@'localhost' IDENTIFIED BY 'Iyad';
3 •  CREATE USER 'manager'@'localhost' IDENTIFIED BY 'Pacizio';
4
5 •  GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
6 •  GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
7
8 •  SHOW GRANTS FOR 'manager'@'localhost';
9
10 •  REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Grants for manager@localhost
GRANT USAGE ON *.* TO `manager`@`localho...
GRANT ALL PRIVILEGES ON `atelier_commerce...

Figure 23 - CREATE USER, GRANT, SHOW GRANTS et REVOKE

Observations et points d'attention

En relisant les résultats, trois points méritent d'être signalés :

- **Requêtes liées aux paiements :** les captures des figures 14, 16 et 18 affichent un résultat vide, et la colonne mode_paiement de la vue (figure 22) vaut NULL. Cela suggère que la table **paiements** était vide au moment de ces requêtes : l'INSERT de la figure 5 n'a probablement pas été exécuté. Un `SELECT * FROM paiements;` permet de le vérifier ; une fois les données insérées, ces requêtes doivent renvoyer des lignes (Alice, Bob et Charlie pour IN et EXISTS ; Dina seule pour NOT IN).
- **Ordre des arguments de DATEDIFF :** DATEDIFF(date1, date2) calcule date1 - date2. En écrivant DATEDIFF(date_paiement, date_commande), le résultat est positif si le paiement suit la commande. Dans les données de test, les dates de paiement sont antérieures aux commandes (par exemple 12/01 pour une commande du 20/01), ce qui donnera des délais négatifs.
- **Données de détail :** la dernière ligne de details_commandes (4,2,3) référence une commande n°4 qui n'existe pas ; avec la clé étrangère, MySQL refuserait cette ligne. Il faut vérifier que l'insertion a bien été exécutée telle qu'attendue.

Conclusion

Ce travail a permis de parcourir l'ensemble du cycle de vie d'une base MySQL : conception du schéma avec clés étrangères, insertion des données, jointures, agrégations, fonctions (IF, DATEDIFF), sous-requêtes, optimisation par index vérifiée avec EXPLAIN, vue, puis gestion des droits avec GRANT, SHOW GRANTS et REVOKE. Toutes les captures demandées dans la consigne de l'atelier sont fournies dans ce rapport.